

Provided proper attribution is provided, Google hereby grants permission to reproduce the tables and figures in this paper solely for use in journalistic or scholarly works.

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com	Noam Shazeer* Google Brain noam@google.com	Niki Parmar* Google Research nikip@google.com	Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com	Aidan N. Gomez*[†] University of Toronto aidan@cs.toronto.edu	Łukasz Kaiser* Google Brain lukaszkaizer@google.com	
Illia Polosukhin*[‡] illia.polosukhin@gmail.com			

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

*Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

[†]Work performed while at Google Brain.

[‡]Work performed while at Google Research.

第 1 页中文解析 (1/1)

[1] 如果提供适当的归属，Google 特此授权复制本文中的表格和图形，仅供新闻或学术作品使用。

[2] arXiv:1706.03762v7 [cs.CL] 2023 年 8 月 2 日。

[3]
主要序列转导模型基于复杂的循环或卷积神经网络，包括编码器和解码器。性能最好的模型还通过注意力机制连接编码器和解码器。我们提出了一种新的简单网络架构——Transformer，它完全基于注意力机制，完全不需要递归和卷积。对两个机器翻译任务的实验表明，这些模型具有卓越的质量，同时具有更高的并行性，并且需要的训练时间显著减少。我们的模型在 WMT 2014 英语到德语翻译任务中取得了 28.4 BLEU，比现有的最佳结果（包括集成）提高了 2 BLEU 以上。在 WMT 2014 英法翻译任务中，我们的模型在 8 个 GPU 上训练 3.5 天后，建立了新的单模型最先进 BLEU 分数 41.8，这只是文献中最佳模型训练成本的一小部分。我们通过将 Transformer 成功应用于具有大量和有限训练数据的英语选区解析，证明 Transformer 可以很好地推广到其他任务。

[4] *同等贡献。列表顺序是随机的。Jakob 提议用自注意力机制取代 RNN，并开始努力评估这个想法。Ashish 与 Illia 一起设计并实现了第一个 Transformer 模型，并至关重要地参与了这项工作的各个方面。诺姆提出了缩放点积注意力、多头注意力和无参数位置表示，并成为参与几乎每个细节的另一个人。Niki 在我们的原始代码库和 tensor2tensor 中设计、实现、调整和评估了无数模型变体。Llion 还尝试了新颖的模型变体，负责我们的初始代码库以及高效的推理和可视化。Lukasz 和 Aidan 花了无数漫长的时间设计和实现 tensor2tensor 的各个部分，替换了我们早期的代码库，极大地改善了结果并极大地加速了我们的研究。† 在 Google Brain 期间完成的工作。‡ 在 Google Research 期间进行的工作。

[5] 第 31 届神经信息处理系统会议 (NIPS 2017)，美国加利福尼亚州长滩。

1 Introduction

Recurrent neural networks, long short-term memory [13] and gated recurrent [7] neural networks in particular, have been firmly established as state of the art approaches in sequence modeling and transduction problems such as language modeling and machine translation [35, 2, 5]. Numerous efforts have since continued to push the boundaries of recurrent language models and encoder-decoder architectures [38, 24, 15].

Recurrent models typically factor computation along the symbol positions of the input and output sequences. Aligning the positions to steps in computation time, they generate a sequence of hidden states h_t , as a function of the previous hidden state h_{t-1} and the input for position t . This inherently sequential nature precludes parallelization within training examples, which becomes critical at longer sequence lengths, as memory constraints limit batching across examples. Recent work has achieved significant improvements in computational efficiency through factorization tricks [21] and conditional computation [32], while also improving model performance in case of the latter. The fundamental constraint of sequential computation, however, remains.

Attention mechanisms have become an integral part of compelling sequence modeling and transduction models in various tasks, allowing modeling of dependencies without regard to their distance in the input or output sequences [2, 19]. In all but a few cases [27], however, such attention mechanisms are used in conjunction with a recurrent network.

In this work we propose the Transformer, a model architecture eschewing recurrence and instead relying entirely on an attention mechanism to draw global dependencies between input and output. The Transformer allows for significantly more parallelization and can reach a new state of the art in translation quality after being trained for as little as twelve hours on eight P100 GPUs.

2 Background

The goal of reducing sequential computation also forms the foundation of the Extended Neural GPU [16], ByteNet [18] and ConvS2S [9], all of which use convolutional neural networks as basic building block, computing hidden representations in parallel for all input and output positions. In these models, the number of operations required to relate signals from two arbitrary input or output positions grows in the distance between positions, linearly for ConvS2S and logarithmically for ByteNet. This makes it more difficult to learn dependencies between distant positions [12]. In the Transformer this is reduced to a constant number of operations, albeit at the cost of reduced effective resolution due to averaging attention-weighted positions, an effect we counteract with Multi-Head Attention as described in section 3.2.

Self-attention, sometimes called intra-attention is an attention mechanism relating different positions of a single sequence in order to compute a representation of the sequence. Self-attention has been used successfully in a variety of tasks including reading comprehension, abstractive summarization, textual entailment and learning task-independent sentence representations [4, 27, 28, 22].

End-to-end memory networks are based on a recurrent attention mechanism instead of sequence-aligned recurrence and have been shown to perform well on simple-language question answering and language modeling tasks [34].

To the best of our knowledge, however, the Transformer is the first transduction model relying entirely on self-attention to compute representations of its input and output without using sequence-aligned RNNs or convolution. In the following sections, we will describe the Transformer, motivate self-attention and discuss its advantages over models such as [17, 18] and [9].

3 Model Architecture

Most competitive neural sequence transduction models have an encoder-decoder structure [5, 2, 35]. Here, the encoder maps an input sequence of symbol representations $(x_1, \dots, x_n)$ to a sequence of continuous representations $\mathbf{z} = (z_1, \dots, z_n)$. Given $\mathbf{z}$, the decoder then generates an output sequence $(y_1, \dots, y_m)$ of symbols one element at a time. At each step the model is auto-regressive [10], consuming the previously generated symbols as additional input when generating the next.

第 2 页中文解析 (1/2)

[1]

循环神经网络，特别是长短期记忆[13]和门控循环[7]神经网络，已被牢固地确立为序列建模和转导问题（例如语言建模和机器翻译）中最先进的方法[35,2,5]。此后，人们做出了许多努力，不断突破循环语言模型和编码器-解码器架构的界限[38,24,15]。

[2]

循环模型通常会沿着输入和输出序列的符号位置进行计算。将位置与计算时间中的步骤对齐，它们生成一系列隐藏状态 h_t ，作为先前隐藏状态 h_{t-1} 和位置 t

输入的函数。这种固有的顺序性质阻碍了训练示例中的并行化，这在较长的序列长度上变得至关重要，因为内存限制限制了示例之间的批处理。最近的工作通过分解技巧 [21] 和条件计算 [32]

显著提高了计算效率，同时还提高了后者的模型性能。然而，顺序计算的基本限制仍然存在。

[3]

注意力机制已经成为各种任务中引人注目的序列建模和转导模型的一个组成部分，允许对依赖关系进行建模，而不考虑它们在输入或输出序列中的距离[2,

19]。然而，除了少数情况外[27]，这种注意力机制都是与循环网络结合使用的。

[4] 在这项工作中，我们提出了

Transformer，这是一种避免重复的模型架构，而是完全依赖注意力机制来绘制输入和输出之间的全局依赖关系。Transformer 允许显著提高并行度，并且在 8 个 P100 GPU 上进行短短 12 小时的训练后，可以在翻译质量方面达到新的水平。

[5] 减少顺序计算的目标也构成了扩展神经 GPU [16]、ByteNet [18] 和 ConvS2S [9]

的基础，所有这些都使用卷积神经网络作为基本构建块，并行计算所有输入和输出位置的隐藏表示。在这些模型中，关联来自两个任意输入或输出位置的信号所需的操作数量随着位置之间的距离而增加，对于 ConvS2S 呈线性增长，对于 ByteNet 呈对数增长。这使得学习遥远位置之间的依赖关系变得更加困难[12]。在

Transformer

中，这被减少到恒定数量的操作，尽管由于平均注意力加权位置而导致有效分辨率降低，我们用多头注意力来抵消这种影响，如第 3.2 节所述。

[6]

自注意力（有时称为内部注意力）是一种将单个序列的不同位置相关联的注意力机制，以便计算序列的表示。自注意力已成功应用于各种任务，包括阅读理解、抽象概括、文本蕴涵和学习任务无关的句子表示[4,27,28,22]。

[7]

端到端记忆网络基于循环注意机制而不是序列对齐循环，并且已被证明在简单语言问答和语言建模任务上表现良好[34]。

[8] 然而，据我们所知，Transformer 是第一个完全依赖自注意力来计算其输入和输出表示而不使用序列对齐

第 2 页中文解析 (2/2)

[9] 大多数竞争性神经序列转导模型都具有编码器-解码器结构 [5,2,35]。这里，编码器将符号表示的输入序列 $(x_1, \dots, x_n)$ 映射到连续表示序列 $z = (z_1, \dots, z_n)$ 。给定 z ，解码器然后每次生成一个元素的符号输出序列 $(y_1, \dots, y_m)$ 。在每个步骤中，模型都是自回归的[10]，在生成下一个时将先前生成的符号用作附加输入。

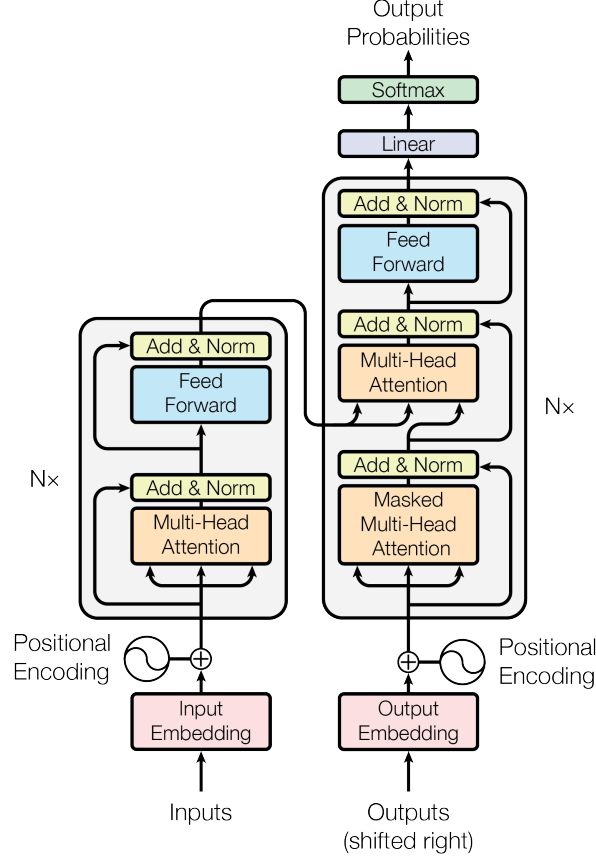


Figure 1: The Transformer - model architecture.

The Transformer follows this overall architecture using stacked self-attention and point-wise, fully connected layers for both the encoder and decoder, shown in the left and right halves of Figure 1, respectively.

3.1 Encoder and Decoder Stacks

Encoder: The encoder is composed of a stack of $N = 6$ identical layers. Each layer has two sub-layers. The first is a multi-head self-attention mechanism, and the second is a simple, position-wise fully connected feed-forward network. We employ a residual connection [11] around each of the two sub-layers, followed by layer normalization [1]. That is, the output of each sub-layer is $\text{LayerNorm}(x + \text{Sublayer}(x))$, where $\text{Sublayer}(x)$ is the function implemented by the sub-layer itself. To facilitate these residual connections, all sub-layers in the model, as well as the embedding layers, produce outputs of dimension $d_{\text{model}} = 512$.

Decoder: The decoder is also composed of a stack of $N = 6$ identical layers. In addition to the two sub-layers in each encoder layer, the decoder inserts a third sub-layer, which performs multi-head attention over the output of the encoder stack. Similar to the encoder, we employ residual connections around each of the sub-layers, followed by layer normalization. We also modify the self-attention sub-layer in the decoder stack to prevent positions from attending to subsequent positions. This masking, combined with fact that the output embeddings are offset by one position, ensures that the predictions for position i can depend only on the known outputs at positions less than i .

3.2 Attention

An attention function can be described as mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors. The output is computed as a weighted sum

第 3 页中文解析 (1/1)

[1] 图 1：Transformer - 模型架构。

[2] Transformer 遵循这一整体架构，对编码器和解码器使用堆叠自注意力和逐点、全连接层，分别如图 1 的左半部分和右半部分所示。

[3] 3.1 编码器和解码器堆栈。

[4] 编码器：编码器由 $N = 6$

个相同层的堆栈组成。每层有两个子层。第一个是多头自注意力机制，第二个是简单的、位置明智的全连接前馈网络。我们在两个子层周围采用残差连接[11]，然后进行层归一化[1]。即每个子层的输出为 $\text{LayerNorm}(x + \text{Sublayer}(x))$ ，其中

$\text{Sublayer}(x)$ 是子层本身实现的函数。为了促进这些残差连接，模型中的所有子层以及嵌入层都会生成维度 $d_{\text{model}} = 512$ 的输出。

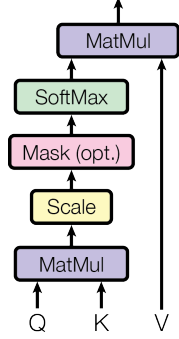
[5] 解码器：解码器也是由 $N = 6$

个相同层的堆栈组成。除了每个编码器层中的两个子层之外，解码器还插入第三个子层，该子层对编码器堆栈的输出执行多头关注。与编码器类似，我们在每个子层周围采用残差连接，然后进行层归一化。我们还修改了解码器堆栈中的自注意力子层，以防止位置关注后续位置。这种掩蔽与输出嵌入偏移一个位置的事实相结合，确保位置 i 的预测只能依赖于小于 i 的位置处的已知输出。

[6]

注意力函数可以描述为将查询和一组键值对映射到输出，其中查询、键、值和输出都是向量。输出被计算为加权和。

Scaled Dot-Product Attention



Multi-Head Attention

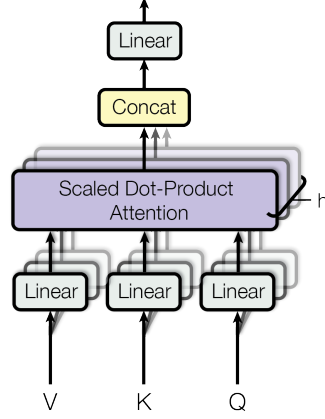


Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key.

3.2.1 Scaled Dot-Product Attention

We call our particular attention "Scaled Dot-Product Attention" (Figure 2). The input consists of queries and keys of dimension d_k , and values of dimension d_v . We compute the dot products of the query with all keys, divide each by $\sqrt{d_k}$, and apply a softmax function to obtain the weights on the values.

In practice, we compute the attention function on a set of queries simultaneously, packed together into a matrix Q . The keys and values are also packed together into matrices K and V . We compute the matrix of outputs as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

The two most commonly used attention functions are additive attention [2], and dot-product (multiplicative) attention. Dot-product attention is identical to our algorithm, except for the scaling factor of $\frac{1}{\sqrt{d_k}}$. Additive attention computes the compatibility function using a feed-forward network with a single hidden layer. While the two are similar in theoretical complexity, dot-product attention is much faster and more space-efficient in practice, since it can be implemented using highly optimized matrix multiplication code.

While for small values of d_k the two mechanisms perform similarly, additive attention outperforms dot product attention without scaling for larger values of d_k [3]. We suspect that for large values of d_k , the dot products grow large in magnitude, pushing the softmax function into regions where it has extremely small gradients⁴. To counteract this effect, we scale the dot products by $\frac{1}{\sqrt{d_k}}$.

3.2.2 Multi-Head Attention

Instead of performing a single attention function with d_{model} -dimensional keys, values and queries, we found it beneficial to linearly project the queries, keys and values h times with different, learned linear projections to d_k , d_k and d_v dimensions, respectively. On each of these projected versions of queries, keys and values we then perform the attention function in parallel, yielding d_v -dimensional

⁴To illustrate why the dot products get large, assume that the components of q and k are independent random variables with mean 0 and variance 1. Then their dot product, $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$, has mean 0 and variance d_k .

第 4 页中文解析 (1/1)

[1] 图 2：（左）缩放的点积注意力。（右）多头注意力由多个并行运行的注意力层组成。

[2] 值的权重，其中分配给每个值的权重是通过查询与相应键的兼容性函数计算的。

[3] 3.2.1 缩放点积注意力。

[4] 我们将我们的特别注意力称为“缩放点积注意力”（图 2）。输入由维度 d_k 的查询和键以及维度 d_v 的值组成。我们使用所有键计算查询的点积，将每个键除以 $\sqrt{d_k}$ ，然后应用 softmax 函数来获取值的权重。

[5] 在实践中，我们同时计算一组查询的注意力函数，将其打包到矩阵 Q 中。键和值也打包到矩阵 K 和 V 中。我们将输出矩阵计算为：

[6] 两种最常用的注意力函数是加性注意力[2]和点积（乘法）注意力。除了缩放因子 $1/\sqrt{d_k}$ 之外，点积注意力与我们的算法相同。附加注意力使用具有单个隐藏层的前馈网络来计算兼容性函数。虽然两者在理论复杂性上相似，但点积注意力在实践中更快、更节省空间，因为它可以使用高度优化的矩阵乘法代码来实现。

[7] 虽然对于较小的 d_k 值，两种机制的表现相似，但加性注意力优于点积注意力，而无需针对较大的 d_k 值进行缩放 [3]。我们怀疑，对于较大的 d_k 值，点积的幅度会变大，从而将 softmax 函数推入梯度极小的区域 4。为了抵消这种影响，我们将点积缩放 $1/\sqrt{d_k}$ 。

[8] 3.2.2 多头注意力。

[9] 我们发现，使用不同的学习线性投影将查询、键和值分别线性投影到 d_k 、 d_k 和 d_v 维度 h 次是有益的，而不是使用 d_{model} 维度的键、值和查询执行单个注意力函数。然后，我们对查询、键和值的每个投影版本并行执行注意力函数，产生 d_v 维。

[10] 4 为了说明点积变大的原因，假设 q 和 k 的分量是均值 0 和方差 1 的独立随机变量。那么它们的点积 $q \cdot k = \sum_{i=1}^d q_i k_i$ 的均值 0 和方差 d 。

output values. These are concatenated and once again projected, resulting in the final values, as depicted in Figure 2.

Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions. With a single attention head, averaging inhibits this.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Where the projections are parameter matrices $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ and $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

In this work we employ $h = 8$ parallel attention layers, or heads. For each of these we use $d_k = d_v = d_{\text{model}}/h = 64$. Due to the reduced dimension of each head, the total computational cost is similar to that of single-head attention with full dimensionality.

3.2.3 Applications of Attention in our Model

The Transformer uses multi-head attention in three different ways:

- In "encoder-decoder attention" layers, the queries come from the previous decoder layer, and the memory keys and values come from the output of the encoder. This allows every position in the decoder to attend over all positions in the input sequence. This mimics the typical encoder-decoder attention mechanisms in sequence-to-sequence models such as [38, 2, 9].
- The encoder contains self-attention layers. In a self-attention layer all of the keys, values and queries come from the same place, in this case, the output of the previous layer in the encoder. Each position in the encoder can attend to all positions in the previous layer of the encoder.
- Similarly, self-attention layers in the decoder allow each position in the decoder to attend to all positions in the decoder up to and including that position. We need to prevent leftward information flow in the decoder to preserve the auto-regressive property. We implement this inside of scaled dot-product attention by masking out (setting to $-\infty$) all values in the input of the softmax which correspond to illegal connections. See Figure 2.

3.3 Position-wise Feed-Forward Networks

In addition to attention sub-layers, each of the layers in our encoder and decoder contains a fully connected feed-forward network, which is applied to each position separately and identically. This consists of two linear transformations with a ReLU activation in between.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (2)$$

While the linear transformations are the same across different positions, they use different parameters from layer to layer. Another way of describing this is as two convolutions with kernel size 1. The dimensionality of input and output is $d_{\text{model}} = 512$, and the inner-layer has dimensionality $d_{ff} = 2048$.

3.4 Embeddings and Softmax

Similarly to other sequence transduction models, we use learned embeddings to convert the input tokens and output tokens to vectors of dimension d_{model} . We also use the usual learned linear transformation and softmax function to convert the decoder output to predicted next-token probabilities. In our model, we share the same weight matrix between the two embedding layers and the pre-softmax linear transformation, similar to [30]. In the embedding layers, we multiply those weights by $\sqrt{d_{\text{model}}}$.

第 5 页中文解析 (1/2)

[1] 输出值。将它们连接并再次投影，得到最终值，如图 2 所示。

[2] 多头注意力允许模型共同关注来自不同位置的不同表示子空间的信息。对于单一注意力头，平均会抑制这种情况。

[3] $\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O$ 。

[4] 其中 $\text{head}_i = \text{Attention}(QW_{Q_i}, KW_{K_i}, VW_{V_i})$ 。

[5] 其中投影是参数矩阵 $W_{Q_i} \in \mathbb{R}^{d_{\text{model}} \times d_k}$ 、 $W_{K_i} \in \mathbb{R}^{d_{\text{model}} \times d_k}$ 、 $W_{V_i} \in \mathbb{R}^{d_{\text{model}} \times d_v}$ 和 $W_O \in \mathbb{R}^{h d_v \times d_{\text{model}}}$ 。

[6] 在这项工作中，我们采用 $h = 8$ 个并行注意力层或头。对于每一个，我们使用 $d_k = d_v = d_{\text{model}}/h = 64$ 。由于每个头的维度减少，总计算成本与全维度的单头注意力相似。

[7] 3.2.3 模型中注意力的应用。

[8] Transformer 以三种不同的方式使用多头注意力：

[9] • 在“编码器-解码器注意”层中，查询来自前一个解码器层，内存键和值来自编码器的输出。这允许解码器中的每个位置都参与输入序列中的所有位置。这模仿了序列到序列模型中典型的编码器-解码器注意机制，例如[38,2,9]。

[10] • 编码器包含自注意力层。在自注意力层中，所有键、值和查询都来自同一位置，在本例中是编码器中前一层的输出。编码器中的每个位置可以关注编码器上一层中的所有位置。

[11] 类似地，解码器中的自注意力层允许解码器中的每个位置关注解码器中直到并包括该位置的所有位置。我们需要防止解码器中的左向信息流以保留自回归属性。我们通过屏蔽（设置为 $-\infty$ ）softmax 输入中与非法连接相对应的所有值来实现缩放点积注意力的内部。参见图 2。

[12] 3.3 位置式前馈网络。

[13] 除了注意力子层之外，我们的编码器和解码器中的每个层都包含一个完全连接的前馈网络，该网络单独且相同地应用于每个位置。这由两个线性变换组成，中间有一个 ReLU 激活。

第 5 页中文解析 (2/2)

另一种描述方式是内核大小为 1 的两个卷积。输入和输出的维度为 $d_{\text{model}} = 512$ ，内层的维度为 $d_{\text{ff}} = 2048$ 。

[16] 3.4 嵌入和 Softmax。

[17] 与其他序列转导模型类似，我们使用学习嵌入将输入标记和输出标记转换为维度 d_{model} 的向量。我们还使用通常学习的线性变换和 softmax

函数将解码器输出转换为预测的下一个令牌概率。在我们的模型中，我们在两个嵌入层和 pre-softmax 线性变换之间共享相同的权重矩阵，类似于[30]。在嵌入层中，我们将这些权重乘以 $\sqrt{d_{\text{model}}}$ 。

Table 1: Maximum path lengths, per-layer complexity and minimum number of sequential operations for different layer types. n is the sequence length, d is the representation dimension, k is the kernel size of convolutions and r the size of the neighborhood in restricted self-attention.

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

3.5 Positional Encoding

Since our model contains no recurrence and no convolution, in order for the model to make use of the order of the sequence, we must inject some information about the relative or absolute position of the tokens in the sequence. To this end, we add "positional encodings" to the input embeddings at the bottoms of the encoder and decoder stacks. The positional encodings have the same dimension d_{model} as the embeddings, so that the two can be summed. There are many choices of positional encodings, learned and fixed [9].

In this work, we use sine and cosine functions of different frequencies:

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

where pos is the position and i is the dimension. That is, each dimension of the positional encoding corresponds to a sinusoid. The wavelengths form a geometric progression from 2π to $10000 \cdot 2\pi$. We chose this function because we hypothesized it would allow the model to easily learn to attend by relative positions, since for any fixed offset k , PE_{pos+k} can be represented as a linear function of PE_{pos} .

We also experimented with using learned positional embeddings [9] instead, and found that the two versions produced nearly identical results (see Table 3 row (E)). We chose the sinusoidal version because it may allow the model to extrapolate to sequence lengths longer than the ones encountered during training.

4 Why Self-Attention

In this section we compare various aspects of self-attention layers to the recurrent and convolutional layers commonly used for mapping one variable-length sequence of symbol representations $(x_1, \dots, x_n)$ to another sequence of equal length $(z_1, \dots, z_n)$, with $x_i, z_i \in \mathbb{R}^d$, such as a hidden layer in a typical sequence transduction encoder or decoder. Motivating our use of self-attention we consider three desiderata.

One is the total computational complexity per layer. Another is the amount of computation that can be parallelized, as measured by the minimum number of sequential operations required.

The third is the path length between long-range dependencies in the network. Learning long-range dependencies is a key challenge in many sequence transduction tasks. One key factor affecting the ability to learn such dependencies is the length of the paths forward and backward signals have to traverse in the network. The shorter these paths between any combination of positions in the input and output sequences, the easier it is to learn long-range dependencies [12]. Hence we also compare the maximum path length between any two input and output positions in networks composed of the different layer types.

As noted in Table 1, a self-attention layer connects all positions with a constant number of sequentially executed operations, whereas a recurrent layer requires $O(n)$ sequential operations. In terms of computational complexity, self-attention layers are faster than recurrent layers when the sequence

第 6 页中文解析 (1/1)

[1] 表 1：不同层类型的最大路径长度、每层复杂度和最小顺序操作数。n 是序列长度，d 是表示维度，k 是卷积核大小，r 是受限自注意力中邻域的大小。

[2] 层类型 每层复杂度 顺序最大路径长度 操作自注意力 $O(n^2 \cdot d)$ $O(1)$ $O(1)$ 循环 $O(n \cdot d^2)$ $O(n)$ $O(n)$ 卷积 $O(k \cdot n \cdot d^2)$ $O(1)$ $O(\log k(n))$ 自注意力（受限） $O(r \cdot n \cdot d)$ $O(1)$ $O(n/r)$ 。

[3]

由于我们的模型不包含递归和卷积，为了使模型能够利用序列的顺序，我们必须注入一些有关序列中标记的相对或绝对位置的信息。为此，我们将“位置编码”添加到编码器和解码器堆栈底部的输入嵌入中。位置编码与嵌入具有相同的维度 d_{model} ，因此可以将两者相加。位置编码有多种选择，有学习的和固定的[9]。

[4] 在这项工作中，我们使用不同频率的正弦和余弦函数：

[5] 其中 pos 是位置，i 是维度。也就是说，位置编码的每个维度对应于正弦曲线。波长形成从 2π 到 $10000 \cdot 2\pi$

的几何级数。我们选择这个函数是因为我们假设它可以让模型轻松学习关注相对位置，因为对于任何固定偏移 k， $PE_{\text{pos}+k}$ 可以表示为 PE_{pos} 的线性函数。

[6] 我们还尝试使用学习的位置嵌入 [9]，发现这两个版本产生几乎相同的结果（参见表 3 行 (E)）。我们选择正弦版本，因为它可以允许模型推断出比训练期间遇到的序列长度更长的序列长度。

[7]

在本节中，我们将自注意力层的各个方面与通常用于将符号表示的一个可变长度序列 $(x_1, \dots, x_n)$ 映射到另一个等长序列 $(z_1, \dots, z_n)$ 的循环层和卷积层进行比较，其中 $x_i, z_i \in \mathbb{R}^d$ ，例如典型序列转导编码器或解码器中的隐藏层。为了激发我们使用自我注意力，我们考虑了三个需求。

[8] 一是每层的总计算复杂度。另一个是可以并行化的计算量，以所需的最小顺序操作数来衡量。

[9]

第三个是网络中远程依赖之间的路径长度。学习远程依赖性在许多序列转导任务中的一个关键挑战。影响学习这种依赖性的能力的一个关键因素是前向和后向信号在网络中必须经过的路径的长度。输入和输出序列中的任意位置组合之间的路径越短，学习远程依赖关系就越容易[12]。因此，我们还比较了由不同层类型组成的网络中任意两个输入和输出位置之间的最大路径长度。

[10] 如表 1 所示，自注意力层通过恒定数量的顺序执行操作连接所有位置，而循环层需要 $O(n)$ 顺序操作。就计算复杂度而言，当序列。

length n is smaller than the representation dimensionality d , which is most often the case with sentence representations used by state-of-the-art models in machine translations, such as word-piece [38] and byte-pair [31] representations. To improve computational performance for tasks involving very long sequences, self-attention could be restricted to considering only a neighborhood of size r in the input sequence centered around the respective output position. This would increase the maximum path length to $O(n/r)$. We plan to investigate this approach further in future work.

A single convolutional layer with kernel width $k < n$ does not connect all pairs of input and output positions. Doing so requires a stack of $O(n/k)$ convolutional layers in the case of contiguous kernels, or $O(\log_k(n))$ in the case of dilated convolutions [18], increasing the length of the longest paths between any two positions in the network. Convolutional layers are generally more expensive than recurrent layers, by a factor of k . Separable convolutions [6], however, decrease the complexity considerably, to $O(k \cdot n \cdot d + n \cdot d^2)$. Even with $k = n$, however, the complexity of a separable convolution is equal to the combination of a self-attention layer and a point-wise feed-forward layer, the approach we take in our model.

As side benefit, self-attention could yield more interpretable models. We inspect attention distributions from our models and present and discuss examples in the appendix. Not only do individual attention heads clearly learn to perform different tasks, many appear to exhibit behavior related to the syntactic and semantic structure of the sentences.

5 Training

This section describes the training regime for our models.

5.1 Training Data and Batching

We trained on the standard WMT 2014 English-German dataset consisting of about 4.5 million sentence pairs. Sentences were encoded using byte-pair encoding [3], which has a shared source-target vocabulary of about 37000 tokens. For English-French, we used the significantly larger WMT 2014 English-French dataset consisting of 36M sentences and split tokens into a 32000 word-piece vocabulary [38]. Sentence pairs were batched together by approximate sequence length. Each training batch contained a set of sentence pairs containing approximately 25000 source tokens and 25000 target tokens.

5.2 Hardware and Schedule

We trained our models on one machine with 8 NVIDIA P100 GPUs. For our base models using the hyperparameters described throughout the paper, each training step took about 0.4 seconds. We trained the base models for a total of 100,000 steps or 12 hours. For our big models, (described on the bottom line of table 3), step time was 1.0 seconds. The big models were trained for 300,000 steps (3.5 days).

5.3 Optimizer

We used the Adam optimizer [20] with $\beta_1 = 0.9$, $\beta_2 = 0.98$ and $\epsilon = 10^{-9}$. We varied the learning rate over the course of training, according to the formula:

$$lrate = d_{\text{model}}^{-0.5} \cdot \min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5}) \quad (3)$$

This corresponds to increasing the learning rate linearly for the first $warmup_steps$ training steps, and decreasing it thereafter proportionally to the inverse square root of the step number. We used $warmup_steps = 4000$.

5.4 Regularization

We employ three types of regularization during training:

第 7 页中文解析 (1/2)

[1] 长度 n 小于表示维度 d ，这最常见于机器翻译中最先进模型使用的句子表示，例如单词片段 [38] 和字节对 [31]

表示。为了提高涉及很长序列的任务的计算性能，自注意力可以限制为仅考虑输入序列中以相应输出位置为中心的大小为 r 的邻域。这会将最大路径长度增加到 $O(n/r)$ 。我们计划在未来的工作中进一步研究这种方法。

[2] 内核宽度 $k < n$ 的单个卷积层不会连接所有输入和输出位置对。这样做需要在连续内核的情况下需要一堆 $O(n/k)$ 卷积层，或者在扩张卷积的情况下需要 $O(\log k(n))$

[18]，从而增加网络中任意两个位置之间的最长路径的长度。卷积层通常比循环层贵 k 倍。然而，可分离卷积

[6] 大大降低了复杂性，达到 $O(k \cdot n \cdot d + n \cdot d^2)$ 。然而，即使 $k =$

n ，可分离卷积的复杂度也等于自注意力层和逐点前馈层的组合，这是我们在模型中采用的方法。

[3]

作为附带好处，自注意力可以产生更多可解释的模型。我们检查模型中的注意力分布，并在附录中展示和讨论示例。个体注意力头不仅清楚地学习执行不同的任务，而且许多注意力头似乎表现出与句子的句法和语义结构相关的行为。

[4] 本节描述我们模型的训练制度。

[5] 5.1 训练数据和批处理。

[6] 我们使用标准 WMT 2014 英语-德语数据集进行训练，该数据集包含约 450

万个句子对。句子使用字节对编码 [3] 进行编码，该编码具有约 37000

个标记的共享源-目标词汇表。对于英语-法语，我们使用了更大的 WMT 2014 英语-法语数据集，该数据集由 3600 万个句子组成，并将标记拆分为 32000 个单词片段词汇表

[38]。句子对按大致序列长度分批在一起。每个训练批次包含一组句子对，其中包含大约 25000 个源标记和 25000 个目标标记。

[7] 5.2 硬件和时间表。

[8] 我们在一台配备 8 个 NVIDIA P100 GPU

的机器上训练我们的模型。对于我们使用整篇论文中描述的超参数的基本模型，每个训练步骤大约需要 0.4 秒。我们对基础模型进行了总计 100,000 步或 12 小时的训练。对于我们的大型模型（如表 3 的底线所述），步长时间为 1.0 秒。大模型接受了 300,000 步（3.5 天）的训练。

[9] 我们使用 Adam 优化器 [20]，其中 $\beta_1 = 0.9$ 、 $\beta_2 = 0.98$ 且 $\epsilon = 10^{-9}$ 。我们根据以下公式在训练过程中改变学习率：

[10] $\text{lr} = d^{-0.5} \cdot \min(\text{step_num}^{-0.5}, \text{step_num} \cdot \text{warmup_steps}^{-1.5})$ (3)。

[11] 这对应于第一个 Warmup steps

[12] 我们在训练期间采用三种类型的正则化：。

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

Residual Dropout We apply dropout [33] to the output of each sub-layer, before it is added to the sub-layer input and normalized. In addition, we apply dropout to the sums of the embeddings and the positional encodings in both the encoder and decoder stacks. For the base model, we use a rate of $P_{drop} = 0.1$.

Label Smoothing During training, we employed label smoothing of value $\epsilon_{ls} = 0.1$ [36]. This hurts perplexity, as the model learns to be more unsure, but improves accuracy and BLEU score.

6 Results

6.1 Machine Translation

On the WMT 2014 English-to-German translation task, the big transformer model (Transformer (big) in Table 2) outperforms the best previously reported models (including ensembles) by more than 2.0 BLEU, establishing a new state-of-the-art BLEU score of 28.4. The configuration of this model is listed in the bottom line of Table 3. Training took 3.5 days on 8 P100 GPUs. Even our base model surpasses all previously published models and ensembles, at a fraction of the training cost of any of the competitive models.

On the WMT 2014 English-to-French translation task, our big model achieves a BLEU score of 41.0, outperforming all of the previously published single models, at less than 1/4 the training cost of the previous state-of-the-art model. The Transformer (big) model trained for English-to-French used dropout rate $P_{drop} = 0.1$, instead of 0.3.

For the base models, we used a single model obtained by averaging the last 5 checkpoints, which were written at 10-minute intervals. For the big models, we averaged the last 20 checkpoints. We used beam search with a beam size of 4 and length penalty $\alpha = 0.6$ [38]. These hyperparameters were chosen after experimentation on the development set. We set the maximum output length during inference to input length + 50, but terminate early when possible [38].

Table 2 summarizes our results and compares our translation quality and training costs to other model architectures from the literature. We estimate the number of floating point operations used to train a model by multiplying the training time, the number of GPUs used, and an estimate of the sustained single-precision floating-point capacity of each GPU ⁵.

6.2 Model Variations

To evaluate the importance of different components of the Transformer, we varied our base model in different ways, measuring the change in performance on English-to-German translation on the

⁵We used values of 2.8, 3.7, 6.0 and 9.5 TFLOPS for K80, K40, M40 and P100, respectively.

第 8 页中文解析 (1/2)

- [1] 表 2：在 2014 年英语到德语和英语到法语 newstest2014 测试中，Transformer 比之前最先进的模型获得了更好的 BLEU 分数，而训练成本仅为一小部分。
- [2] EN-DE EN-FR EN-DE EN-FR ByteNet [18] 23.75 Deep-Att + PosUnk [39] 39.2 1.0 · 1020。
- [3] Deep-Att + PosUnk 合奏 [39] 40.4 8.0 · 1020。
- [4] GNMT + RL 系综 [38] 26.30 41.16 1.8 · 1020 1.1 · 1021。
- [5] ConvS2S 集成 [9] 26.36 41.29 7.7 · 1019 1.2 · 1021。
- [6] 变压器（基本型号） 27.3 38.1 3.3 · 1018。
- [7] 变压器（大） 28.4 41.8 2.3 · 1019。
- [8] 残差 Dropout 我们将 dropout [33] 应用于每个子层的输出，然后将其添加到子层输入并进行归一化。此外，我们将 dropout 应用于编码器和解码器堆栈中的嵌入和位置编码的总和。对于基本模型，我们使用 $P_{drop} = 0.1$ 的比率。
- [9] 标签平滑在训练过程中，我们采用了值 $\epsilon_{ls} = 0.1$ 的标签平滑[36]。这会降低模型的困惑度，因为模型会变得更加不确定，但会提高准确性和 BLEU 分数。
- [10] 在 WMT 2014 英语到德语翻译任务中，大型 Transformer 模型（表 2 中的 Transformer (big)）比之前报告的最佳模型（包括集成）的 BLEU 高出 2.0 以上，建立了新的最先进的 BLEU 分数 28.4。该模型的配置列于表 3 的最后一行。在 8 个 P100 GPU 上训练花费了 3.5 天。甚至我们的基础模型也超越了之前发布的所有模型和集成，而训练成本只是任何竞争模型的一小部分。
- [11] 在 WMT 2014 英法翻译任务中，我们的大模型获得了 41.0 的 BLEU 分数，优于之前发布的所有单一模型，而训练成本不到之前最先进模型的 1/4。针对英语到法语训练的 Transformer（大）模型使用的辍学率 $P_{drop} = 0.1$ ，而不是 0.3。
- [12] 对于基础模型，我们使用通过对最后 5 个检查点进行平均而获得的单个模型，这些检查点以 10 分钟的间隔写入。对于大型模型，我们对最后 20 个检查点进行了平均。我们使用波束搜索，波束大小为 4，长度惩罚 $\alpha = 0.6$ [38]。这些超参数是在开发集上进行实验后选择的。我们将推理期间的最大输出长度设置为输入长度+50，但尽可能提前终止[38]。
- [13] 表 2 总结了我们的结果，并将我们的翻译质量和训练成本与文献中的其他模型架构进行了比较。我们通过将训练时间、使用的 GPU 数量以及每个 GPU 5

第 8 页中文解析 (2/2)

[14] 为了评估 Transformer

不同组件的重要性，我们以不同的方式改变了我们的基本模型，测量了英语到德语翻译的性能变化。

[15] 5 我们对 K80、K40、M40 和 P100 分别使用 2.8、3.7、6.0 和 9.5 TFLOPS 值。

Table 3: Variations on the Transformer architecture. Unlisted values are identical to those of the base model. All metrics are on the English-to-German translation development set, newstest2013. Listed perplexities are per-wordpiece, according to our byte-pair encoding, and should not be compared to per-word perplexities.

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	train steps	PPL (dev)	BLEU (dev)	params $\times 10^6$		
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65		
(A)					1	512				5.29	24.9			
					4	128				5.00	25.5			
					16	32				4.91	25.8			
					32	16				5.01	25.4			
(B)					16					5.16	25.1	58		
					32					5.01	25.4	60		
(C)	2									6.11	23.7	36		
	4									5.19	25.3	50		
	8									4.88	25.5	80		
	256				32	32					5.75	24.5	28	
	1024				128	128					4.66	26.0	168	
			1024									5.12	25.4	53
			4096									4.75	26.2	90
(D)							0.0			5.77	24.6			
							0.2			4.95	25.5			
								0.0		4.67	25.3			
								0.2		5.47	25.7			
(E)	positional embedding instead of sinusoids									4.92	25.7			
big	6	1024	4096	16				0.3	300K	4.33	26.4	213		

development set, newstest2013. We used beam search as described in the previous section, but no checkpoint averaging. We present these results in Table 3.

In Table 3 rows (A), we vary the number of attention heads and the attention key and value dimensions, keeping the amount of computation constant, as described in Section 3.2.2. While single-head attention is 0.9 BLEU worse than the best setting, quality also drops off with too many heads.

In Table 3 rows (B), we observe that reducing the attention key size d_k hurts model quality. This suggests that determining compatibility is not easy and that a more sophisticated compatibility function than dot product may be beneficial. We further observe in rows (C) and (D) that, as expected, bigger models are better, and dropout is very helpful in avoiding over-fitting. In row (E) we replace our sinusoidal positional encoding with learned positional embeddings [9], and observe nearly identical results to the base model.

6.3 English Constituency Parsing

To evaluate if the Transformer can generalize to other tasks we performed experiments on English constituency parsing. This task presents specific challenges: the output is subject to strong structural constraints and is significantly longer than the input. Furthermore, RNN sequence-to-sequence models have not been able to attain state-of-the-art results in small-data regimes [37].

We trained a 4-layer transformer with $d_{\text{model}} = 1024$ on the Wall Street Journal (WSJ) portion of the Penn Treebank [25], about 40K training sentences. We also trained it in a semi-supervised setting, using the larger high-confidence and BerkleyParser corpora from with approximately 17M sentences [37]. We used a vocabulary of 16K tokens for the WSJ only setting and a vocabulary of 32K tokens for the semi-supervised setting.

We performed only a small number of experiments to select the dropout, both attention and residual (section 5.4), learning rates and beam size on the Section 22 development set, all other parameters remained unchanged from the English-to-German base translation model. During inference, we

- [1] 表 3：Transformer 架构的变化。未列出的值与基本型号的值相同。所有指标均来自英德翻译开发集 newstest2013。根据我们的字节对编码，列出的困惑度是每个单词的困惑度，不应与每个单词的困惑度进行比较。
- [2] $N \cdot d_{model} \cdot d_{ff} \cdot h \cdot d_k \cdot d_v \cdot P_{drop} \cdot \sum_{l=1}^L \text{train PPL BLEU params 步骤} \cdot (\text{dev}) \cdot (\text{dev}) \times 106$ 。
- [3] 0.0 5.77 24.6 0.2 4.95 25.5 0.0 4.67 25.3 0.2 5.47 25.7 (E) 位置嵌入代替正弦曲线 4.92 25.7 big 6 1024 4096 16 0.3 300K 4.33 26.4 213。
- [4] 开发集，newstest2013。我们使用了上一节中描述的波束搜索，但没有检查点平均。我们在表 3 中列出了这些结果。
- [5] 在表 3 行 (A) 中，我们改变注意力头的数量以及注意力键和值维度，保持计算量恒定，如第 3.2.2 节所述。虽然单头注意力比最佳设置差 0.9 BLEU，但头数过多质量也会下降。
- [6] 在表 3 行 (B) 中，我们观察到减少注意力密钥大小 d_k 会损害模型质量。这表明确定兼容性并不容易，并且比点积更复杂的兼容性函数可能是有益的。我们在 (C) 和 (D) 行中进一步观察到，正如预期的那样，模型越大越好，并且 dropout 对于避免过度拟合非常有帮助。在行 (E) 中，我们用学习的位置嵌入替换正弦位置编码 [9]，并观察到与基本模型几乎相同的结果。
- [7] 6.3 英语选区解析。
- [8] 为了评估 Transformer 是否可以推广到其他任务，我们对英语选区解析进行了实验。这项任务提出了具体的挑战：输出受到强大的结构约束，并且明显长于输入。此外，RNN 序列到序列模型尚未能够在小数据情况下获得最先进的结果 [37]。
- [9] 我们在 Penn Treebank [25] 的《华尔街日报》(WSJ) 部分训练了一个 $d_{model} = 1024$ 的 4 层 Transformer，大约 40K 训练句子。我们还在半监督环境中对其进行了训练，使用了更大的高置信度和 BerkleyParser 语料库，其中包含大约 1700 万个句子 [37]。我们在仅《华尔街日报》设置中使用了 16K 个标记的词汇表，在半监督设置中使用了 32K 个标记的词汇表。
- [10] 我们仅进行了少量实验来选择第 22 节开发集上的 dropout、注意力和残差（第 5.4 节）、学习率和波束大小，所有其他参数与英语到德语的基础翻译模型保持不变。在推理过程中，我们。

Table 4: The Transformer generalizes well to English constituency parsing (Results are on Section 23 of WSJ)

Parser	Training	WSJ 23 F1
Vinyals & Kaiser et al. (2014) [37]	WSJ only, discriminative	88.3
Petrov et al. (2006) [29]	WSJ only, discriminative	90.4
Zhu et al. (2013) [40]	WSJ only, discriminative	90.4
Dyer et al. (2016) [8]	WSJ only, discriminative	91.7
Transformer (4 layers)	WSJ only, discriminative	91.3
Zhu et al. (2013) [40]	semi-supervised	91.3
Huang & Harper (2009) [14]	semi-supervised	91.3
McClosky et al. (2006) [26]	semi-supervised	92.1
Vinyals & Kaiser et al. (2014) [37]	semi-supervised	92.1
Transformer (4 layers)	semi-supervised	92.7
Luong et al. (2015) [23]	multi-task	93.0
Dyer et al. (2016) [8]	generative	93.3

increased the maximum output length to input length + 300. We used a beam size of 21 and $\alpha = 0.3$ for both WSJ only and the semi-supervised setting.

Our results in Table 4 show that despite the lack of task-specific tuning our model performs surprisingly well, yielding better results than all previously reported models with the exception of the Recurrent Neural Network Grammar [8].

In contrast to RNN sequence-to-sequence models [37], the Transformer outperforms the Berkeley-Parser [29] even when training only on the WSJ training set of 40K sentences.

7 Conclusion

In this work, we presented the Transformer, the first sequence transduction model based entirely on attention, replacing the recurrent layers most commonly used in encoder-decoder architectures with multi-headed self-attention.

For translation tasks, the Transformer can be trained significantly faster than architectures based on recurrent or convolutional layers. On both WMT 2014 English-to-German and WMT 2014 English-to-French translation tasks, we achieve a new state of the art. In the former task our best model outperforms even all previously reported ensembles.

We are excited about the future of attention-based models and plan to apply them to other tasks. We plan to extend the Transformer to problems involving input and output modalities other than text and to investigate local, restricted attention mechanisms to efficiently handle large inputs and outputs such as images, audio and video. Making generation less sequential is another research goals of ours.

The code we used to train and evaluate our models is available at <https://github.com/tensorflow/tensor2tensor>.

Acknowledgements We are grateful to Nal Kalchbrenner and Stephan Gouws for their fruitful comments, corrections and inspiration.

References

- [1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *CoRR*, abs/1409.0473, 2014.
- [3] Denny Britz, Anna Goldie, Minh-Thang Luong, and Quoc V. Le. Massive exploration of neural machine translation architectures. *CoRR*, abs/1703.03906, 2017.
- [4] Jianpeng Cheng, Li Dong, and Mirella Lapata. Long short-term memory-networks for machine reading. *arXiv preprint arXiv:1601.06733*, 2016.

第 10 页中文解析 (1/2)

[1] 表 4：Transformer 很好地推广了英语选区解析（结果见《华尔街日报》第 23 节）。

[2] 解析器培训 WSJ 23 F1 Vinyals 和 Kaiser 等。(2014) [37] 仅《华尔街日报》，歧视性 88.3 Petrov 等人。(2006) [29] 仅《华尔街日报》，歧视性 90.4 Zhu 等人。(2013) [40] 仅《华尔街日报》，歧视性 90.4 Dyer 等人。(2016) [8] 仅 WSJ，判别性 91.7 Transformer (4 层) 仅 WSJ，判别性 91.3 Zhu 等人。(2013) [40] 半监督 91.3 Huang & Harper (2009) [14] 半监督 91.3 McClosky 等人。(2006) [26] 半监督 92.1 Vinyals & Kaiser et al。(2014) [37] 半监督 92.1 Transformer (4 层) 半监督 92.7 Luong 等人。(2015) [23] 多任务 93.0 Dyer 等人。(2016) [8] 生成 93.3。

[3] 将最大输出长度增加到输入长度 + 300。对于仅 WSJ 和半监督设置，我们使用了 21 的波束大小和 $\alpha = 0.3$ 。

[4] 表 4

中的结果表明，尽管缺乏特定于任务的调整，我们的模型表现得令人惊讶地好，比之前报告的所有模型（循环神经网络语法除外）[8]产生了更好的结果。

[5] 与 RNN 序列到序列模型 [37] 相比，即使仅在 WSJ 40K 句子训练集上进行训练，Transformer 的性能也优于 Berkeley-Parser [29]。

[6] 在这项工作中，我们提出了

Transformer，这是第一个完全基于注意力的序列转换模型，用多头自注意力取代了编码器-解码器架构中最常用的循环层。

[7] 对于翻译任务，Transformer 的训练速度明显快于基于循环层或卷积层的架构。在 WMT 2014 英语-德语和 WMT 2014

英语-法语翻译任务中，我们都达到了新的最先进水平。在前一项任务中，我们最好的模型甚至优于所有先前报告的集成。

[8] 我们对基于注意力的模型的未来感到兴奋，并计划将其应用于其他任务。我们计划将 Transformer 扩展到涉及文本以外的输入和输出模式的问题，并研究局部的、受限的注意力机制，以有效地处理图像、音频和视频等大型输入和输出。减少一代的顺序是我们的另一个研究目标。

[9] 我们用于训练和评估模型的代码可在 <https://github.com/tensorflow/tensor2tensor> 上找到。

[10] 致谢我们感谢 Nal Kalchbrenner 和 Stephan Gouws 富有成效的评论、更正和启发。

[11] [1] 吉米·雷巴、杰米·瑞安·基罗斯和杰弗里·E·辛顿。层标准化。arXiv 预印本 arXiv:1607.06450, 2016。

[12] [2] Dzmitry Bahdanau、Kyunghyun Cho 和 Yoshua

第 10 页中文解析 (2/2)

对神经机器翻译架构的大规模探索。 CoRR, abs/1703.03906, 2017。

[14] [4] 程建鹏, 李东, Mirella Lapata。用于机器阅读的长短期记忆网络。 arXiv 预印本 arXiv:1601.06733, 2016。

- [5] Kyunghyun Cho, Bart van Merriënboer, Çağlar Gülçehre, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *CoRR*, abs/1406.1078, 2014.
- [6] Francois Chollet. Xception: Deep learning with depthwise separable convolutions. *arXiv preprint arXiv:1610.02357*, 2016.
- [7] Junyoung Chung, Çağlar Gülçehre, Kyunghyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. *CoRR*, abs/1412.3555, 2014.
- [8] Chris Dyer, Adhiguna Kuncoro, Miguel Ballesteros, and Noah A. Smith. Recurrent neural network grammars. In *Proc. of NAACL*, 2016.
- [9] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N. Dauphin. Convolutional sequence to sequence learning. *arXiv preprint arXiv:1705.03122v2*, 2017.
- [10] Alex Graves. Generating sequences with recurrent neural networks. *arXiv preprint arXiv:1308.0850*, 2013.
- [11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.
- [12] Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi, and Jürgen Schmidhuber. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies, 2001.
- [13] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [14] Zhongqiang Huang and Mary Harper. Self-training PCFG grammars with latent annotations across languages. In *Proceedings of the 2009 Conference on Empirical Methods in Natural Language Processing*, pages 832–841. ACL, August 2009.
- [15] Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu. Exploring the limits of language modeling. *arXiv preprint arXiv:1602.02410*, 2016.
- [16] Łukasz Kaiser and Samy Bengio. Can active memory replace attention? In *Advances in Neural Information Processing Systems, (NIPS)*, 2016.
- [17] Łukasz Kaiser and Ilya Sutskever. Neural GPUs learn algorithms. In *International Conference on Learning Representations (ICLR)*, 2016.
- [18] Nal Kalchbrenner, Lasse Espeholt, Karen Simonyan, Aaron van den Oord, Alex Graves, and Koray Kavukcuoglu. Neural machine translation in linear time. *arXiv preprint arXiv:1610.10099v2*, 2017.
- [19] Yoon Kim, Carl Denton, Luong Hoang, and Alexander M. Rush. Structured attention networks. In *International Conference on Learning Representations*, 2017.
- [20] Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.
- [21] Oleksii Kuchaiev and Boris Ginsburg. Factorization tricks for LSTM networks. *arXiv preprint arXiv:1703.10722*, 2017.
- [22] Zhouhan Lin, Minwei Feng, Cicero Nogueira dos Santos, Mo Yu, Bing Xiang, Bowen Zhou, and Yoshua Bengio. A structured self-attentive sentence embedding. *arXiv preprint arXiv:1703.03130*, 2017.
- [23] Minh-Thang Luong, Quoc V. Le, Ilya Sutskever, Oriol Vinyals, and Łukasz Kaiser. Multi-task sequence to sequence learning. *arXiv preprint arXiv:1511.06114*, 2015.
- [24] Minh-Thang Luong, Hieu Pham, and Christopher D Manning. Effective approaches to attention-based neural machine translation. *arXiv preprint arXiv:1508.04025*, 2015.

第 11 页中文解析 (1/2)

[1] [5] Kyunghyun Cho、Bart van Merriënboer、Caglar Gulcehre、Fethi Bougares、Holger Schwenk 和 Yoshua Bengio。使用 rnn 编码器-解码器学习短语表示以进行统计机器翻译。CoRR, abs/1406.1078, 2014。

[2] [6] 弗朗索瓦·乔莱。Xception：具有深度可分离卷积的深度学习。arXiv 预印本 arXiv:1610.02357, 2016。

[3] [7] Junyoung Chung、Çaglar Gülçehre、Kyunghyun Cho 和 Yoshua Bengio。门控循环神经网络对序列建模的实证评估。CoRR, abs/1412.3555, 2014。

[4] [8] 克里斯·戴尔、阿迪古纳·昆科罗、米格尔·巴列斯特罗斯和诺亚·A·史密斯。递归神经网络语法。在过程中。NAACL, 2016 年。

[5] [9] 乔纳斯·格林、迈克尔·奥利、大卫·格兰吉尔、丹尼斯·亚拉茨和扬·N·多芬。卷积序列到序列学习。arXiv 预印本 arXiv:1705.03122v2, 2017。

[6] [10] 亚历克斯·格雷夫斯。使用循环神经网络生成序列。arXiv 预印本 arXiv:1308.0850, 2013。

[7] [11] 何凯明, 张翔宇, 任少清, 孙健。用于图像识别的深度残差学习。IEEE 计算机视觉和模式识别会议论文集, 第 770-778 页, 2016 年。

[8] [12] Sepp Hochreiter、Yoshua Bengio、Paolo Frasconi 和 Jürgen Schmidhuber。循环网络中的梯度流：学习长期依赖性的困难, 2001 年。

[9] [13] 塞普·霍赫赖特和于尔根·施米德胡贝尔。长短期记忆。神经计算, 9(8):1735–1780, 1997。

[10] [14] 黄忠强, 玛丽·哈珀。具有跨语言潜在注释的自训练 PCFG 语法。2009 年自然语言处理经验方法会议记录, 第 832-841 页。ACL, 2009 年 8 月。

[11] [15] Rafal Jozefowicz、Oriol Vinyals、Mike Schuster、Noam Shazeer 和 Yonghui Wu。探索语言建模的局限性。arXiv 预印本 arXiv:1602.02410, 2016。

[12] [16] 武卡斯·凯撒和萨米·本吉奥。主动记忆可以取代注意力吗？神经信息处理系统进展, (NIPS), 2016 年。

[13] [17] 武卡斯·凯撒和伊利亚·苏茨克维尔。神经 GPU 学习算法。国际学习表征会议 (ICLR), 2016 年。

[14] [18] Nal Kalchbrenner、Lasse Espeholt、Karen Simonyan、Aaron van den Oord、Alex Graves 和 Koray Kavukcuoglu。线性时间内的神经机器翻译。arXiv 预印本 arXiv:1610.10099v2, 2017。

[16] [20] 迪德里克·金马和吉米·巴。Adam：一种随机优化方法。ICLR, 2015 年。

[17] [21] 奥莱克西·库恰耶夫和鲍里斯·金斯伯格。LSTM 网络的因式分解技巧。arXiv 预印本 arXiv:1703.10722, 2017。

[18] [22] 林周涵, 冯民伟, 西塞罗·诺盖拉·多斯桑托斯, 莫宇, 项兵, 周博文, Yoshua Bengio。结构化的自我关注句子嵌入。arXiv 预印本 arXiv:1703.03130, 2017。

[19] [23] Minh-Thang Luong、Quoc V. Le、Ilya Sutskever、Oriol Vinyals 和 Lukasz Kaiser。多任务序列到序列学习。arXiv 预印本 arXiv:1511.06114, 2015。

[20] [24] Minh-Thang Luong、Hieu Pham 和 Christopher D Manning。基于注意力的神经机器翻译的有效方法。arXiv 预印本 arXiv:1508.04025, 2015。

- [25] Mitchell P Marcus, Mary Ann Marcinkiewicz, and Beatrice Santorini. Building a large annotated corpus of english: The penn treebank. *Computational linguistics*, 19(2):313–330, 1993.
- [26] David McClosky, Eugene Charniak, and Mark Johnson. Effective self-training for parsing. In *Proceedings of the Human Language Technology Conference of the NAACL, Main Conference*, pages 152–159. ACL, June 2006.
- [27] Ankur Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. A decomposable attention model. In *Empirical Methods in Natural Language Processing*, 2016.
- [28] Romain Paulus, Caiming Xiong, and Richard Socher. A deep reinforced model for abstractive summarization. *arXiv preprint arXiv:1705.04304*, 2017.
- [29] Slav Petrov, Leon Barrett, Romain Thibaux, and Dan Klein. Learning accurate, compact, and interpretable tree annotation. In *Proceedings of the 21st International Conference on Computational Linguistics and 44th Annual Meeting of the ACL*, pages 433–440. ACL, July 2006.
- [30] Ofir Press and Lior Wolf. Using the output embedding to improve language models. *arXiv preprint arXiv:1608.05859*, 2016.
- [31] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. *arXiv preprint arXiv:1508.07909*, 2015.
- [32] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- [33] Nitish Srivastava, Geoffrey E Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.
- [34] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. In C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems 28*, pages 2440–2448. Curran Associates, Inc., 2015.
- [35] Ilya Sutskever, Oriol Vinyals, and Quoc VV Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems*, pages 3104–3112, 2014.
- [36] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. *CoRR*, abs/1512.00567, 2015.
- [37] Vinyals & Kaiser, Koo, Petrov, Sutskever, and Hinton. Grammar as a foreign language. In *Advances in Neural Information Processing Systems*, 2015.
- [38] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google’s neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.
- [39] Jie Zhou, Ying Cao, Xuguang Wang, Peng Li, and Wei Xu. Deep recurrent models with fast-forward connections for neural machine translation. *CoRR*, abs/1606.04199, 2016.
- [40] Muhua Zhu, Yue Zhang, Wenliang Chen, Min Zhang, and Jingbo Zhu. Fast and accurate shift-reduce constituent parsing. In *Proceedings of the 51st Annual Meeting of the ACL (Volume 1: Long Papers)*, pages 434–443. ACL, August 2013.

第 12 页中文解析 (1/2)

[1] [25]

米切尔·P·马库斯、玛丽·安·马尔辛凯维奇和比阿特丽斯·圣托里尼。建立一个大型带注释的英语语料库：宾夕法尼亚树库。计算语言学, 19(2):313–330, 1993。

[2] [26] 大卫·麦克洛斯基、尤金·查尼亚克和马克·约翰逊。有效的解析自我训练。NAACL 人类语言技术会议记录, 主会议, 第 152-159 页。ACL, 2006 年 6 月。

[3] [27] Ankur Parikh、Oscar Täckström、Dipanjan Das 和 Jakob Uszkoreit。可分解的注意力模型。自然语言处理的经验方法, 2016 年。

[4] [28] 罗曼·保卢斯, 蔡明熊, 理查德·索彻。用于抽象概括的深度强化模型。arXiv 预印本 arXiv:1705.04304, 2017。

[5] [29] 斯拉夫·彼得罗夫、莱昂·巴雷特、罗曼·蒂博和丹·克莱因。学习准确、紧凑且可解释的树注释。第 21 届国际计算语言学会议和 ACL 第 44 届年会论文集, 第 433-440 页。ACL, 2006 年 7 月。

[6] [30] 奥菲尔出版社和利奥尔·沃尔夫。使用输出嵌入来改进语言模型。arXiv 预印本 arXiv:1608.05859, 2016。

[7] [31] Rico Sennrich、Barry Haddow 和 Alexandra Birch。具有子词单元的稀有词的神经机器翻译。arXiv 预印本 arXiv:1508.07909, 2015。

[8] [32]

诺姆·沙泽尔、阿扎利亚·米尔霍塞尼、克日什托夫·马齐亚兹、安迪·戴维斯、夸克·勒、杰弗里·辛顿和杰夫·迪恩。极其庞大的神经网络：稀疏门控的专家混合层。arXiv 预印本 arXiv:1701.06538, 2017。

[9] [33] Nitish Srivastava、Geoffrey E Hinton、Alex Krizhevsky、Ilya Sutskever 和 Ruslan Salakhutdinov。

Dropout：防止神经网络过度拟合的简单方法。机器学习研究杂志, 15(1)：1929–1958, 2014。

[10] [34] Sainbayar Sukhbaatar、Arthur Szlam、Jason Weston 和 Rob Fergus。端到端内存网络。C. Cortes、N. D. Lawrence、D. D. Lee、M. Sugiyama 和 R. Garnett, 编辑, 《神经信息处理系统进展》第 28 页, 第 2440-2448 页。Curran Associates, Inc., 2015。

[11] [35] Ilya Sutskever、Oriol Vinyals 和 Quoc VV

Le。使用神经网络进行序列到序列学习。神经信息处理系统进展, 第 3104-3112 页, 2014 年。

[12] [36] Christian Szegedy、Vincent Vanhoucke、Sergey Ioffe、Jonathon Shlens 和 Zbigniew Wojna。重新思考计算机视觉的初始架构。CoRR, abs/1512.00567, 2015。

第 12 页中文解析 (2/2)

[14] [38] 吴永辉, Mike Schuster, 陈志峰, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, 曹元, 高勤, Klaus Macherey, 等。谷歌的神经机器翻译系统：弥合人类和机器翻译之间的差距。 arXiv 预印本 arXiv:1609.08144, 2016。

[15] [39] 周杰, 曹英, 王旭光, 李鹏, 徐伟。具有用于神经机器翻译的快进连接的深度循环模型。 CoRR, abs/1606.04199, 2016。

[16] [40] 朱慕华, 张悦, 陈文良, 张敏, 朱静波。快速准确的移位归约成分解析。 ACL 第 51 届年会记录 (第一卷: 长论文), 第 434-443 页。 ACL, 2013 年 8 月。

Attention Visualizations

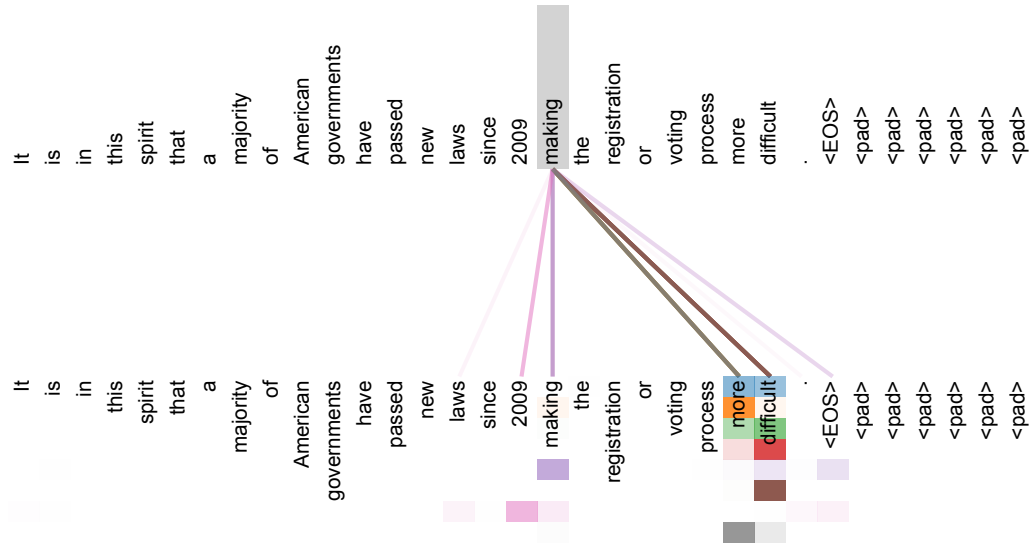


Figure 3: An example of the attention mechanism following long-distance dependencies in the encoder self-attention in layer 5 of 6. Many of the attention heads attend to a distant dependency of the verb ‘making’, completing the phrase ‘making...more difficult’. Attentions here shown only for the word ‘making’. Different colors represent different heads. Best viewed in color.

第 13 页中文解析 (1/1)

[1] 图 3：第 5 层（共 6

层）中编码器自注意力中的长距离依赖关系的注意力机制示例。许多注意力头关注动词“making”的远距离依赖，从而完成短语“making...more

困难”。这里仅显示“制作”一词的注意事项。不同的颜色代表不同的头。最好以彩色形式观看。

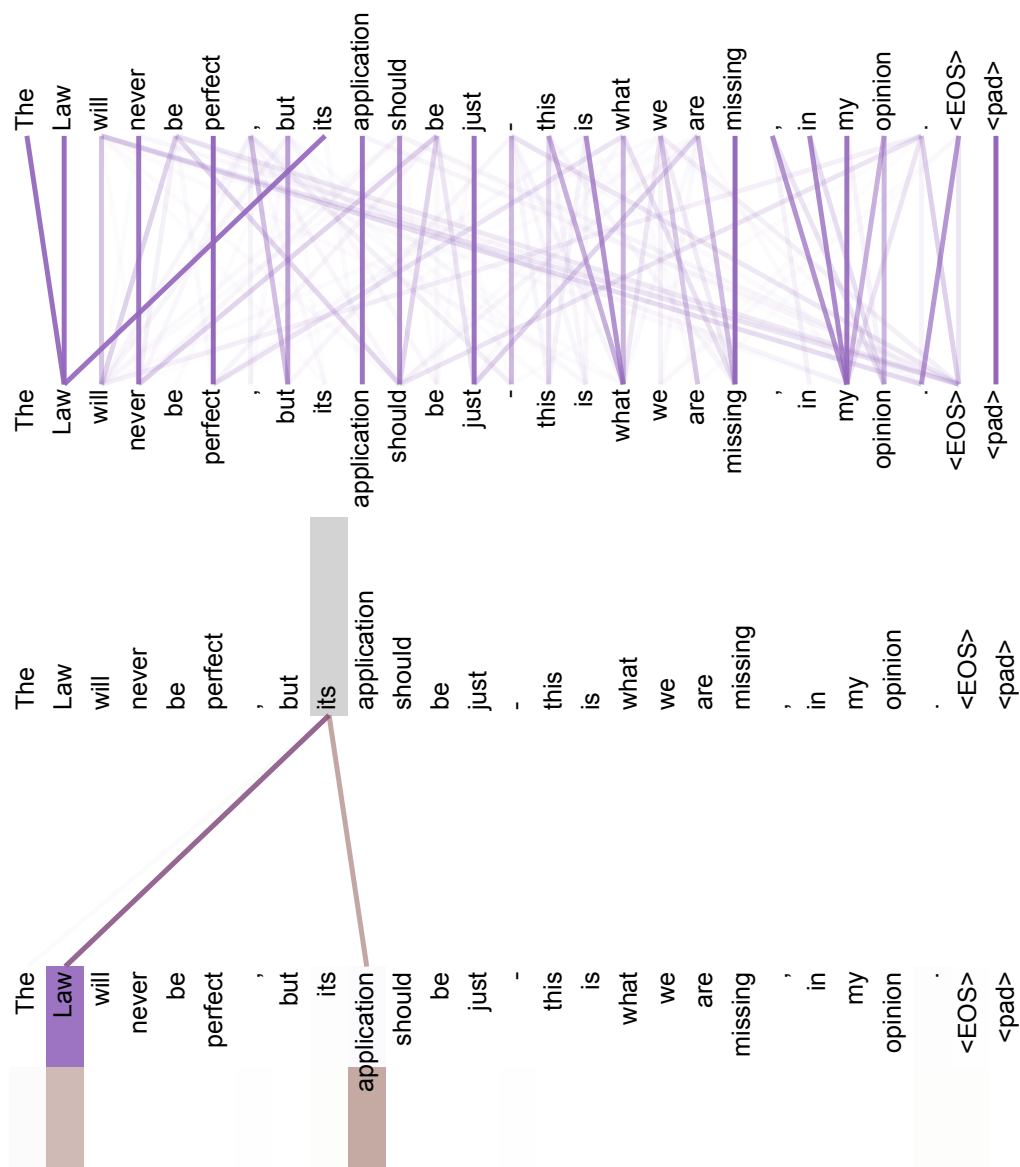


Figure 4: Two attention heads, also in layer 5 of 6, apparently involved in anaphora resolution. Top: Full attentions for head 5. Bottom: Isolated attentions from just the word 'its' for attention heads 5 and 6. Note that the attentions are very sharp for this word.

第 14 页中文解析 (1/1)

[1] 图 4：两个注意力头，也在第 6 层的第 5 层，显然参与了照应解析。顶部：头部 5 的完整注意力。底部：注意力头部 5 和 6 的注意力与单词“its”隔离。请注意，该单词的注意力非常尖锐。

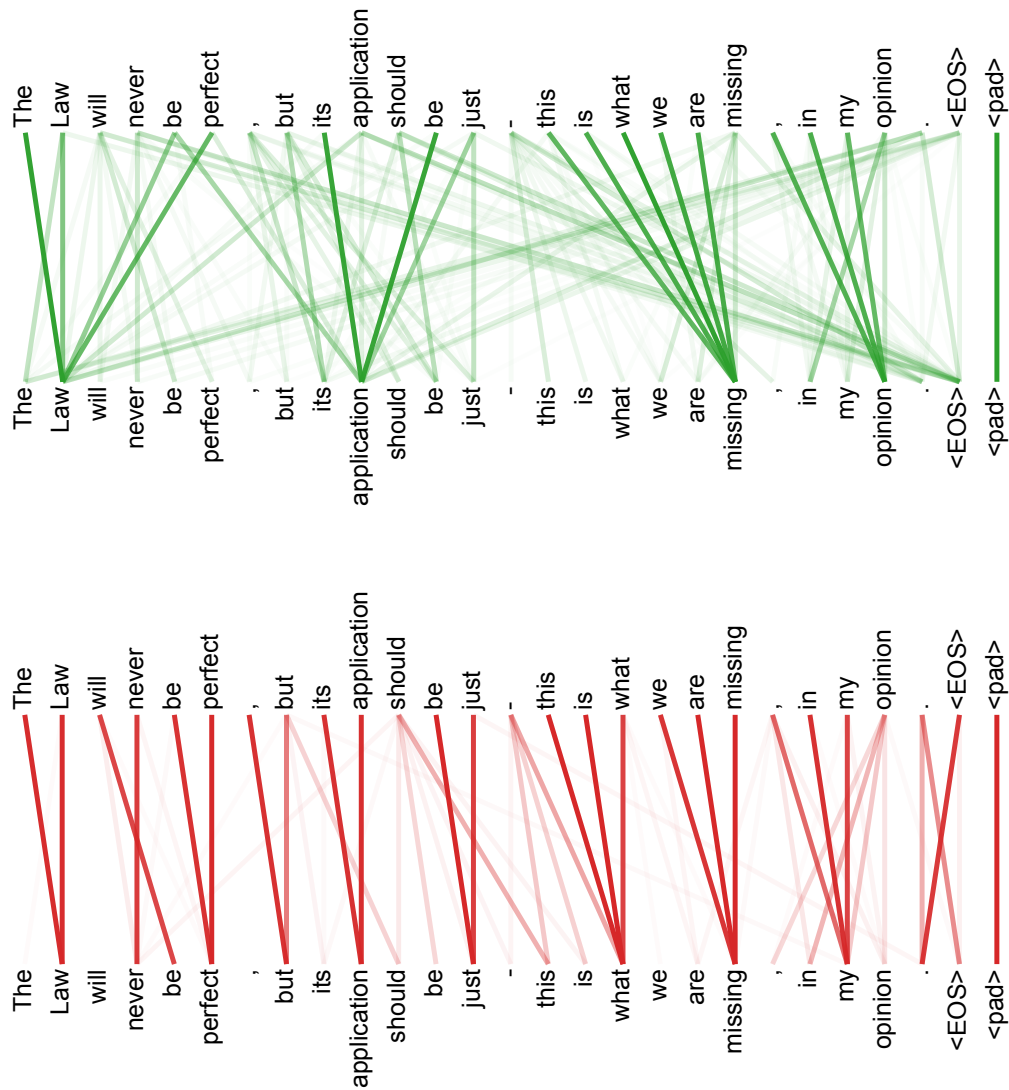


Figure 5: Many of the attention heads exhibit behaviour that seems related to the structure of the sentence. We give two such examples above, from two different heads from the encoder self-attention at layer 5 of 6. The heads clearly learned to perform different tasks.

[1] 图 5：许多注意力头表现出的行为似乎与句子的结构相关。我们在上面给出了两个这样的例子，来自第 5 层或第 6 层编码器自注意力的两个不同的头。这些头显然学会了执行不同的任务。